Zip Code Group Project 1.0

Generated by Doxygen 1.18.0

1 Class Index 1

1.1 Class List . . . 1

2 File Index 3

2.1 File List . . . 3

3 Class Documentation 5

3.1 Buffer Class Reference . . . 5

3.1.1 Detailed Description . . . 5

3.1.2 Constructor & Destructor Documentation . . . 6

3.1.2.1 Buffer() . . . 6

3.1.3 Member Function Documentation . . . 6

3.1.3.1 next() . . . 6

3.1.3.2 readFields() . . . 7

3.1.4 Member Data Documentation . . . 8

3.1.4.1 input . . . 8

3.1.4.2 recordNumber . . . 8

3.2 State Struct Reference . . . 8

3.2.1 Detailed Description . . . 9

3.2.2 Constructor & Destructor Documentation . . . 9

3.2.2.1 State() . . . 9

3.2.3 Member Function Documentation . . . 9

3.2.3.1 update() . . . 9

3.2.4 Member Data Documentation . . . 10

3.2.4.1 east . . . 10

3.2.4.2 north . . . 10

3.2.4.3 south . . . 10

3.2.4.4 west . . . 10

3.3 ZipCodeRecord Struct Reference . . . 10

3.3.1 Detailed Description . . . 11

3.3.2 Member Data Documentation . . . 11

3.3.2.1 latitude . . . 11

3.3.2.2 longitude . . . 11

3.3.2.3 place . . . 11

3.3.2.4 state . . . 12

3.3.2.5 zip . . . 12

4 File Documentation 13

4.1 Buffer.cpp File Reference . . . 13

4.1.1 Detailed Description . . . 13

4.2 Buffer.cpp . . . 13

4.3 Buffer.h File Reference . . . 14

4.3.1 Detailed Description . . . 15

4.4 Buffer.h 15
4.5 Main.cpp File Reference 15
4.5.1 Detailed Description 16
4.5.2 Function Documentation 16
4.5.2.1 main() 16
4.6 Main.cpp 17
4.7 State.h File Reference 17
4.7.1 Detailed Description 17
4.8 State.h 18

Index **19**

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Buffer
: Reads CSV records sequentially, keeping only the current row. Supports quoted fields, escaped quotes, CRLF, and multiline quoted headers. The required column order is ZIP, place, state, county, latitude, longitude 5

State
: Stores the current extreme records for a state. Longitude follows the assignment literally: east = minimum, west = maximum. Coordinate ties select the smallest ZIP so input order cannot change output 8

ZipCodeRecord
: One ZIP code record gathered from a CSV row 10

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

Buffer.cpp
CSV buffering, parsing, and record validation 13
Buffer.h
Sequential CSV input buffer and ZIP code record definition 14
Main.cpp
Gather sequential ZIP records and print alphabetical state extremes 15
State.h
Four directional ZIP code candidates per state 17

Chapter 3

Class Documentation

3.1 Buffer Class Reference

Reads CSV records sequentially, keeping only the current row. Supports quoted fields, escaped quotes, CRLF, and multiline quoted headers. The required column order is ZIP, place, state, county, latitude, longitude.

`#include <Buffer.h>`

Public Member Functions

- Buffer (const std::string &filename)

  Open input and consume one logical header record.
- bool next (ZipCodeRecord &record)

  Gather the next ZIP record from the sequential stream.

Private Member Functions

- bool readFields (std::vector< std::string > &fields)

  Read one logical CSV record.

Private Attributes

- std::ifstream input

  Source stream.
- std::size_t recordNumber = 0

  Logical CSV record number, including header.

3.1.1 Detailed Description

Reads CSV records sequentially, keeping only the current row. Supports quoted fields, escaped quotes, CRLF, and multiline quoted headers. The required column order is ZIP, place, state, county, latitude, longitude.

Definition at line 23 of file Buffer.h.

3.1.2 Constructor & Destructor Documentation

3.1.2.1 Buffer()

```
Buffer::Buffer (
            const std::string & filename)  [explicit]
```

Open input and consume one logical header record.

Parameters

filename	CSV path.

Exceptions

std::runtime_error	If input or header is invalid.

Definition at line 41 of file Buffer.cpp.

```
00041                                       : input(filename) {
00042     if (!input) throw std::runtime_error("Cannot open CSV: " + filename);
00043     // Remove an optional UTF-8 BOM before reading the logical header.
00044     if (input.peek() == 0xEF) {
00045         char bom[3]; input.read(bom, 3);
00046         if (!input || static_cast<unsigned char>(bom[1]) != 0xBB ||
00047             static_cast<unsigned char>(bom[2]) != 0xBF)
00048             throw std::runtime_error("Invalid UTF-8 BOM");
00049     }
00050     std::vector<std::string> header;
00051     if (!readFields(header) || header.size() != 6)
00052         throw std::runtime_error("Expected six CSV header columns");
00053     for (auto& name : header) {
00054         name.erase(std::remove_if(name.begin(), name.end(), [](unsigned char c) { return
      std::isspace(c); }), name.end());
00055         std::transform(name.begin(), name.end(), name.begin(), [](unsigned char c) { return
      std::tolower(c); });
00056     }
00057     const std::vector<std::string> expected{"zipcode", "placename", "state", "county", "lat", "long"};
00058     if (header != expected) throw std::runtime_error("Unexpected CSV column order or names");
00059 }
```

3.1.3 Member Function Documentation

3.1.3.1 next()

```
bool Buffer::next (
            ZipCodeRecord & record)
```

Gather the next ZIP record from the sequential stream.

Parameters

record	Destination ZIP record.

Returns

False at end of file, otherwise true.

Exceptions

std::runtime_error	For malformed rows or coordinates.

Definition at line 61 of file Buffer.cpp.

```
00061                                       {
00062     std::vector<std::string> fields;
00063     while (readFields(fields)) {
00064         if (fields.size() == 1 && fields[0].empty()) continue;
00065         try {
00066             if (fields.size() != 6) throw std::runtime_error("Expected six fields");
00067             record.zip = fields[0];
00068             if (record.zip.empty() || record.zip.size() > 5 ||
00069                 !std::all_of(record.zip.begin(), record.zip.end(), [](unsigned char c) { return
      std::isdigit(c); }))
00070                 throw std::runtime_error("Invalid ZIP code");
00071             record.zip.insert(0, 5 - record.zip.size(), '0');
00072             record.place = fields[1]; record.state = fields[2];
00073             if (record.state.size() != 2) throw std::runtime_error("Invalid state ID");
00074             std::size_t used;
00075             record.latitude = std::stod(fields[4], &used);
00076             if (used != fields[4].size() || !std::isfinite(record.latitude) ||
00077                 record.latitude < -90 || record.latitude > 90)
00078                 throw std::runtime_error("Invalid latitude");
00079             record.longitude = std::stod(fields[5], &used);
00080             if (used != fields[5].size() || !std::isfinite(record.longitude) ||
00081                 record.longitude < -180 || record.longitude > 180)
00082                 throw std::runtime_error("Invalid longitude");
00083         } catch (const std::exception& e) {
00084             throw std::runtime_error("CSV record " + std::to_string(recordNumber) + ": " + e.what());
00085         }
00086         return true;
00087     }
00088     return false;
00089 }
```

3.1.3.2 readFields()

```
bool Buffer::readFields (
            std::vector< std::string > & fields)  [private]
```

Read one logical CSV record.

Parameters

fields	Destination for parsed fields.

Returns

False at end of file; true when a record is available.

Definition at line 10 of file Buffer.cpp.

```
00010                                                 {
00011     fields.clear();
00012     std::string field;
00013     bool quoted = false, closedQuote = false, started = false;
00014     char c;
00015     while (input.get(c)) {
00016         started = true;
00017         if (quoted) {
00018             if (c == '"') {
00019                 if (input.peek() == '"') { input.get(c); field += '"'; }
00020                 else { quoted = false; closedQuote = true; }
```

```
00021             } else { field += c; }
00022         } else if (c == ',') {
00023             fields.push_back(field); field.clear(); closedQuote = false;
00024         } else if (c == '\n' || c == '\r') {
00025             if (c == '\r' && input.peek() == '\n') input.get(c);
00026             fields.push_back(field); ++recordNumber; return true;
00027         } else if (c == '"' && field.empty() && !closedQuote) {
00028             quoted = true;
00029         } else {
00030             if (closedQuote || c == '"')
00031                 throw std::runtime_error("Invalid CSV quoting near record " +
      std::to_string(recordNumber + 1));
00032             field += c;
00033         }
00034     }
00035     if (input.bad()) throw std::runtime_error("Failed while reading CSV");
00036     if (quoted) throw std::runtime_error("Unterminated quoted CSV field");
00037     if (!started) return false;
00038     fields.push_back(field); ++recordNumber; return true;
00039 }
```

3.1.4 Member Data Documentation

3.1.4.1 input

`std::ifstream Buffer::input  [private]`

Source stream.

Definition at line 24 of file Buffer.h.

3.1.4.2 recordNumber

`std::size_t Buffer::recordNumber = 0  [private]`

Logical CSV record number, including header.

Definition at line 25 of file Buffer.h.

The documentation for this class was generated from the following files:

- Buffer.h
- Buffer.cpp

3.2 State Struct Reference

Stores the current extreme records for a state. Longitude follows the assignment literally: east = minimum, west = maximum. Coordinate ties select the smallest ZIP so input order cannot change output.

`#include <State.h>`

Public Member Functions

- State (const ZipCodeRecord &record)

  Initialize all candidates using the state's first record.
- void update (const ZipCodeRecord &record)

  Compare a record with all four current extremes.

Public Attributes

- ZipCodeRecord east
  Minimum longitude candidate.
- ZipCodeRecord west
  Maximum longitude candidate.
- ZipCodeRecord north
  Maximum latitude candidate.
- ZipCodeRecord south
  Minimum latitude candidate.

3.2.1 Detailed Description

Stores the current extreme records for a state. Longitude follows the assignment literally: east = minimum, west = maximum. Coordinate ties select the smallest ZIP so input order cannot change output.

Definition at line 12 of file State.h.

3.2.2 Constructor & Destructor Documentation

3.2.2.1 State()

```
State::State (
            const ZipCodeRecord & record)  [inline], [explicit]
```

Initialize all candidates using the state's first record.

Parameters

record	Initial ZIP record.

Definition at line 20 of file State.h.

```
00020 : east(record), west(record), north(record), south(record) {}
```

3.2.3 Member Function Documentation

3.2.3.1 update()

```
void State::update (
            const ZipCodeRecord & record)  [inline]
```

Compare a record with all four current extremes.

Parameters

record	New ZIP record for this state.

Definition at line 24 of file State.h.

```
00024                                          {
00025         // ZIP tie-breaking is deterministic across different CSV row orders.
00026         if (record.longitude < east.longitude || (record.longitude == east.longitude && record.zip <
      east.zip)) east = record;
00027         if (record.longitude > west.longitude || (record.longitude == west.longitude && record.zip <
      west.zip)) west = record;
00028         if (record.latitude > north.latitude || (record.latitude == north.latitude && record.zip <
      north.zip)) north = record;
00029         if (record.latitude < south.latitude || (record.latitude == south.latitude && record.zip <
      south.zip)) south = record;
00030     }
```

3.2.4 Member Data Documentation

3.2.4.1 east

`ZipCodeRecord State::east`

Minimum longitude candidate.

Definition at line 13 of file State.h.

3.2.4.2 north

`ZipCodeRecord State::north`

Maximum latitude candidate.

Definition at line 15 of file State.h.

3.2.4.3 south

`ZipCodeRecord State::south`

Minimum latitude candidate.

Definition at line 16 of file State.h.

3.2.4.4 west

`ZipCodeRecord State::west`

Maximum longitude candidate.

Definition at line 14 of file State.h.

The documentation for this struct was generated from the following file:

- State.h

3.3 ZipCodeRecord Struct Reference

One ZIP code record gathered from a CSV row.

`#include <Buffer.h>`

Public Attributes

- std::string zip

  Five-digit ZIP code, preserving leading zeros.
- std::string place

  Place name.
- std::string state

  Two-character state or territory ID.
- double latitude = 0

  Latitude in degrees.
- double longitude = 0

  Longitude exactly as supplied in the CSV.

3.3.1 Detailed Description

One ZIP code record gathered from a CSV row.

Definition at line 11 of file Buffer.h.

3.3.2 Member Data Documentation

3.3.2.1 latitude

```
double ZipCodeRecord::latitude = 0
```

Latitude in degrees.

Definition at line 15 of file Buffer.h.

3.3.2.2 longitude

```
double ZipCodeRecord::longitude = 0
```

Longitude exactly as supplied in the CSV.

Definition at line 16 of file Buffer.h.

3.3.2.3 place

```
std::string ZipCodeRecord::place
```

Place name.

Definition at line 13 of file Buffer.h.

3.3.2.4 state

```
std::string ZipCodeRecord::state
```

Two-character state or territory ID.

Definition at line 14 of file Buffer.h.

3.3.2.5 zip

```
std::string ZipCodeRecord::zip
```

Five-digit ZIP code, preserving leading zeros.

Definition at line 12 of file Buffer.h.

The documentation for this struct was generated from the following file:

- Buffer.h

Chapter 4

File Documentation

4.1 Buffer.cpp File Reference

CSV buffering, parsing, and record validation.

```
#include "Buffer.h"
#include <algorithm>
#include <cctype>
#include <cmath>
#include <stdexcept>
```

4.1.1 Detailed Description

CSV buffering, parsing, and record validation.

Definition in file Buffer.cpp.

4.2 Buffer.cpp

Go to the documentation of this file.

```
00001
00004 #include "Buffer.h"
00005 #include <algorithm>
00006 #include <cctype>
00007 #include <cmath>
00008 #include <stdexcept>
00009
00010 bool Buffer::readFields(std::vector<std::string>& fields) {
00011     fields.clear();
00012     std::string field;
00013     bool quoted = false, closedQuote = false, started = false;
00014     char c;
00015     while (input.get(c)) {
00016         started = true;
00017         if (quoted) {
00018             if (c == '"') {
00019                 if (input.peek() == '"') { input.get(c); field += '"'; }
00020                 else { quoted = false; closedQuote = true; }
00021             } else { field += c; }
00022         } else if (c == ',') {
00023             fields.push_back(field); field.clear(); closedQuote = false;
00024         } else if (c == '\n' || c == '\r') {
```

```
00025             if (c == '\r' && input.peek() == '\n') input.get(c);
00026             fields.push_back(field); ++recordNumber; return true;
00027         } else if (c == '"' && field.empty() && !closedQuote) {
00028             quoted = true;
00029         } else {
00030             if (closedQuote || c == '"')
00031                 throw std::runtime_error("Invalid CSV quoting near record " +
      std::to_string(recordNumber + 1));
00032             field += c;
00033         }
00034     }
00035     if (input.bad()) throw std::runtime_error("Failed while reading CSV");
00036     if (quoted) throw std::runtime_error("Unterminated quoted CSV field");
00037     if (!started) return false;
00038     fields.push_back(field); ++recordNumber; return true;
00039 }
00040
00041 Buffer::Buffer(const std::string& filename) : input(filename) {
00042     if (!input) throw std::runtime_error("Cannot open CSV: " + filename);
00043     // Remove an optional UTF-8 BOM before reading the logical header.
00044     if (input.peek() == 0xEF) {
00045         char bom[3]; input.read(bom, 3);
00046         if (!input || static_cast<unsigned char>(bom[1]) != 0xBB ||
00047             static_cast<unsigned char>(bom[2]) != 0xBF)
00048             throw std::runtime_error("Invalid UTF-8 BOM");
00049     }
00050     std::vector<std::string> header;
00051     if (!readFields(header) || header.size() != 6)
00052         throw std::runtime_error("Expected six CSV header columns");
00053     for (auto& name : header) {
00054         name.erase(std::remove_if(name.begin(), name.end(), [](unsigned char c) { return
      std::isspace(c); }), name.end());
00055         std::transform(name.begin(), name.end(), name.begin(), [](unsigned char c) { return
      std::tolower(c); });
00056     }
00057     const std::vector<std::string> expected{"zipcode", "placename", "state", "county", "lat", "long"};
00058     if (header != expected) throw std::runtime_error("Unexpected CSV column order or names");
00059 }
00060
00061 bool Buffer::next(ZipCodeRecord& record) {
00062     std::vector<std::string> fields;
00063     while (readFields(fields)) {
00064         if (fields.size() == 1 && fields[0].empty()) continue;
00065         try {
00066             if (fields.size() != 6) throw std::runtime_error("Expected six fields");
00067             record.zip = fields[0];
00068             if (record.zip.empty() || record.zip.size() > 5 ||
00069                 !std::all_of(record.zip.begin(), record.zip.end(), [](unsigned char c) { return
      std::isdigit(c); }))
00070                 throw std::runtime_error("Invalid ZIP code");
00071             record.zip.insert(0, 5 - record.zip.size(), '0');
00072             record.place = fields[1]; record.state = fields[2];
00073             if (record.state.size() != 2) throw std::runtime_error("Invalid state ID");
00074             std::size_t used;
00075             record.latitude = std::stod(fields[4], &used);
00076             if (used != fields[4].size() || !std::isfinite(record.latitude) ||
00077                 record.latitude < -90 || record.latitude > 90)
00078                 throw std::runtime_error("Invalid latitude");
00079             record.longitude = std::stod(fields[5], &used);
00080             if (used != fields[5].size() || !std::isfinite(record.longitude) ||
00081                 record.longitude < -180 || record.longitude > 180)
00082                 throw std::runtime_error("Invalid longitude");
00083         } catch (const std::exception& e) {
00084             throw std::runtime_error("CSV record " + std::to_string(recordNumber) + ": " + e.what());
00085         }
00086         return true;
00087     }
00088     return false;
00089 }
```

4.3 Buffer.h File Reference

Sequential CSV input buffer and ZIP code record definition.

```
#include <fstream>
#include <string>
#include <vector>
```

Classes

- struct ZipCodeRecord

  One ZIP code record gathered from a CSV row.
- class Buffer

  Reads CSV records sequentially, keeping only the current row. Supports quoted fields, escaped quotes, CRLF, and multiline quoted headers. The required column order is ZIP, place, state, county, latitude, longitude.

4.3.1 Detailed Description

Sequential CSV input buffer and ZIP code record definition.

Definition in file Buffer.h.

4.4 Buffer.h

Go to the documentation of this file.

```
00001
00004 #ifndef ZIP_BUFFER_H
00005 #define ZIP_BUFFER_H
00006 #include <fstream>
00007 #include <string>
00008 #include <vector>
00009
00011 struct ZipCodeRecord {
00012     std::string zip;
00013     std::string place;
00014     std::string state;
00015     double latitude = 0;
00016     double longitude = 0;
00017 };
00018
00023 class Buffer {
00024     std::ifstream input;
00025     std::size_t recordNumber = 0;
00030     bool readFields(std::vector<std::string>& fields);
00031 public:
00036     explicit Buffer(const std::string& filename);
00042     bool next(ZipCodeRecord& record);
00043 };
00044 #endif
```

4.5 Main.cpp File Reference

Gather sequential ZIP records and print alphabetical state extremes.

```
#include "Buffer.h"
#include "State.h"
#include <iomanip>
#include <iostream>
#include <map>
```

Functions

- int main (int argc, char ∗argv[ ])

  Run the ZIP code application.

4.5.1 Detailed Description

Gather sequential ZIP records and print alphabetical state extremes.

Definition in file Main.cpp.

4.5.2 Function Documentation

4.5.2.1 main()

```
int main (
            int argc,
            char * argv[])
```

Run the ZIP code application.

Parameters

argc	Argument count.
argv	Program name and optional input CSV path.

Returns

Zero on success, one for input errors, two for incorrect usage.

Definition at line 15 of file Main.cpp.

```
00015                                {
00016     if (argc > 2) { std::cerr « "Usage: " « argv[0] « " [input.csv]\n"; return 2; }
00017     try {
00018         Buffer buffer(argc == 2 ? argv[1] : "us_postal_codes.csv");
00019         std::map<std::string, State> states;
00020         ZipCodeRecord record;
00021         // Each iteration gathers one record; the complete input is not stored.
00022         while (buffer.next(record)) {
00023             auto it = states.find(record.state);
00024             if (it == states.end()) states.emplace(record.state, State(record));
00025             else it->second.update(record);
00026         }
00027         if (states.empty()) throw std::runtime_error("CSV contains no ZIP records");
00028         std::cout « std::left « std::setw(8) « "State"
00029                   « std::setw(14) « "Easternmost" « std::setw(14) « "Westernmost"
00030                   « std::setw(14) « "Northernmost" « "Southernmost\n";
00031         for (const auto& entry : states) {
00032             const auto& state = entry.second;
00033             std::cout « std::setw(8) « entry.first
00034                       « std::setw(14) « state.east.zip « std::setw(14) « state.west.zip
00035                       « std::setw(14) « state.north.zip « state.south.zip « '\n';
00036         }
00037     } catch (const std::exception& e) { std::cerr « "Error: " « e.what() « '\n'; return 1; }
00038     return 0;
00039 }
```

4.6 Main.cpp

Go to the documentation of this file.

```
00001
00004 #include "Buffer.h"
00005 #include "State.h"
00006 #include <iomanip>
00007 #include <iostream>
00008 #include <map>
00009
00015 int main(int argc, char* argv[]) {
00016     if (argc > 2) { std::cerr « "Usage: " « argv[0] « " [input.csv]\n"; return 2; }
00017     try {
00018         Buffer buffer(argc == 2 ? argv[1] : "us_postal_codes.csv");
00019         std::map<std::string, State> states;
00020         ZipCodeRecord record;
00021         // Each iteration gathers one record; the complete input is not stored.
00022         while (buffer.next(record)) {
00023             auto it = states.find(record.state);
00024             if (it == states.end()) states.emplace(record.state, State(record));
00025             else it->second.update(record);
00026         }
00027         if (states.empty()) throw std::runtime_error("CSV contains no ZIP records");
00028         std::cout « std::left « std::setw(8) « "State"
00029                   « std::setw(14) « "Easternmost" « std::setw(14) « "Westernmost"
00030                   « std::setw(14) « "Northernmost" « "Southernmost\n";
00031         for (const auto& entry : states) {
00032             const auto& state = entry.second;
00033             std::cout « std::setw(8) « entry.first
00034                       « std::setw(14) « state.east.zip « std::setw(14) « state.west.zip
00035                       « std::setw(14) « state.north.zip « state.south.zip « '\n';
00036         }
00037     } catch (const std::exception& e) { std::cerr « "Error: " « e.what() « '\n'; return 1; }
00038     return 0;
00039 }
```

4.7 State.h File Reference

Four directional ZIP code candidates per state.

```
#include "Buffer.h"
```

Classes

- struct State

  Stores the current extreme records for a state. Longitude follows the assignment literally: east = minimum, west = maximum. Coordinate ties select the smallest ZIP so input order cannot change output.

4.7.1 Detailed Description

Four directional ZIP code candidates per state.

Definition in file State.h.

4.8 State.h

Go to the documentation of this file.

```
00001
00004 #ifndef ZIP_STATE_H
00005 #define ZIP_STATE_H
00006 #include "Buffer.h"
00007
00012 struct State {
00013     ZipCodeRecord east;
00014     ZipCodeRecord west;
00015     ZipCodeRecord north;
00016     ZipCodeRecord south;
00020     explicit State(const ZipCodeRecord& record) : east(record), west(record), north(record),
      south(record) {}
00024     void update(const ZipCodeRecord& record) {
00025         // ZIP tie-breaking is deterministic across different CSV row orders.
00026         if (record.longitude < east.longitude || (record.longitude == east.longitude && record.zip <
      east.zip)) east = record;
00027         if (record.longitude > west.longitude || (record.longitude == west.longitude && record.zip <
      west.zip)) west = record;
00028         if (record.latitude > north.latitude || (record.latitude == north.latitude && record.zip <
      north.zip)) north = record;
00029         if (record.latitude < south.latitude || (record.latitude == south.latitude && record.zip <
      south.zip)) south = record;
00030     }
00031 };
00032 #endif
```

Index

Buffer, 5
    Buffer, 6
    input, 8
    next, 6
    readFields, 7
    recordNumber, 8
Buffer.cpp, 13
Buffer.h, 14

east
    State, 10

input
    Buffer, 8

latitude
    ZipCodeRecord, 11
longitude
    ZipCodeRecord, 11

main
    Main.cpp, 16
Main.cpp, 15
    main, 16

next
    Buffer, 6
north
    State, 10

place
    ZipCodeRecord, 11

readFields
    Buffer, 7
recordNumber
    Buffer, 8

south
    State, 10
State, 8
    east, 10
    north, 10
    south, 10
    State, 9
    update, 9
    west, 10
state
    ZipCodeRecord, 11
State.h, 17

update
    State, 9

west
    State, 10

zip
    ZipCodeRecord, 12
ZipCodeRecord, 10
    latitude, 11
    longitude, 11
    place, 11
    state, 11
    zip, 12